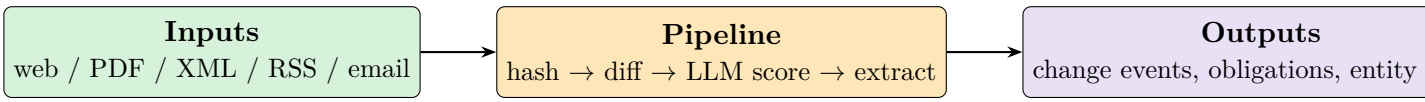
Regulatory Watch

Product & Architecture Overview

What it does

Continuously ingests regulatory content from a variety of sources (web, PDF, XML, RSS, email), detects meaningful changes, scores them with an LLM, and extracts structured obligations — deadlines, actors, actions — into a queryable database.



Project: `regulation-prj-v1` · Stack: Python 3.11, FastAPI, Celery, PostgreSQL 16, Redis 7
LLM: OpenAI `gpt-4o-mini` (JSON mode, temperature 0)
Date: April 17, 2026

Contents

1 The problem **2**

2 High-level architecture **2**
2.1 Component map 2
2.2 Three layers, one pipeline 3

3 Data model **3**
3.1 Relationships (simplified) 3

4 The pipeline in detail **3**
4.1 Stage 1 — Ingestion (L1) 3
4.2 Stage 2 — Versioning & change detection (L2) 4
4.3 Stage 3 — Significance scoring (L3) 4
4.4 Stage 4 — Obligations & entity index (L4) 4

5 Resilience & observability **5**

6 API & operational surface **5**
6.1 REST API (FastAPI, `/docs`) 5
6.2 Celery tasks (auto-scheduled by Celery-beat) 5
6.3 Operator CLIs 6

7 Tech stack **6**

8 Economics **6**

9 Current status **6**

1 The problem

Regulated businesses (banks, importers, pharma, data-processors) have to continuously track rule changes across dozens of authorities, each publishing through different channels (websites, PDFs, RSS feeds, XML bulk dumps, email bulletins). Today this work is done either by armies of analysts reading PDFs, or by brittle scripts that break the moment a web page is rebranded.

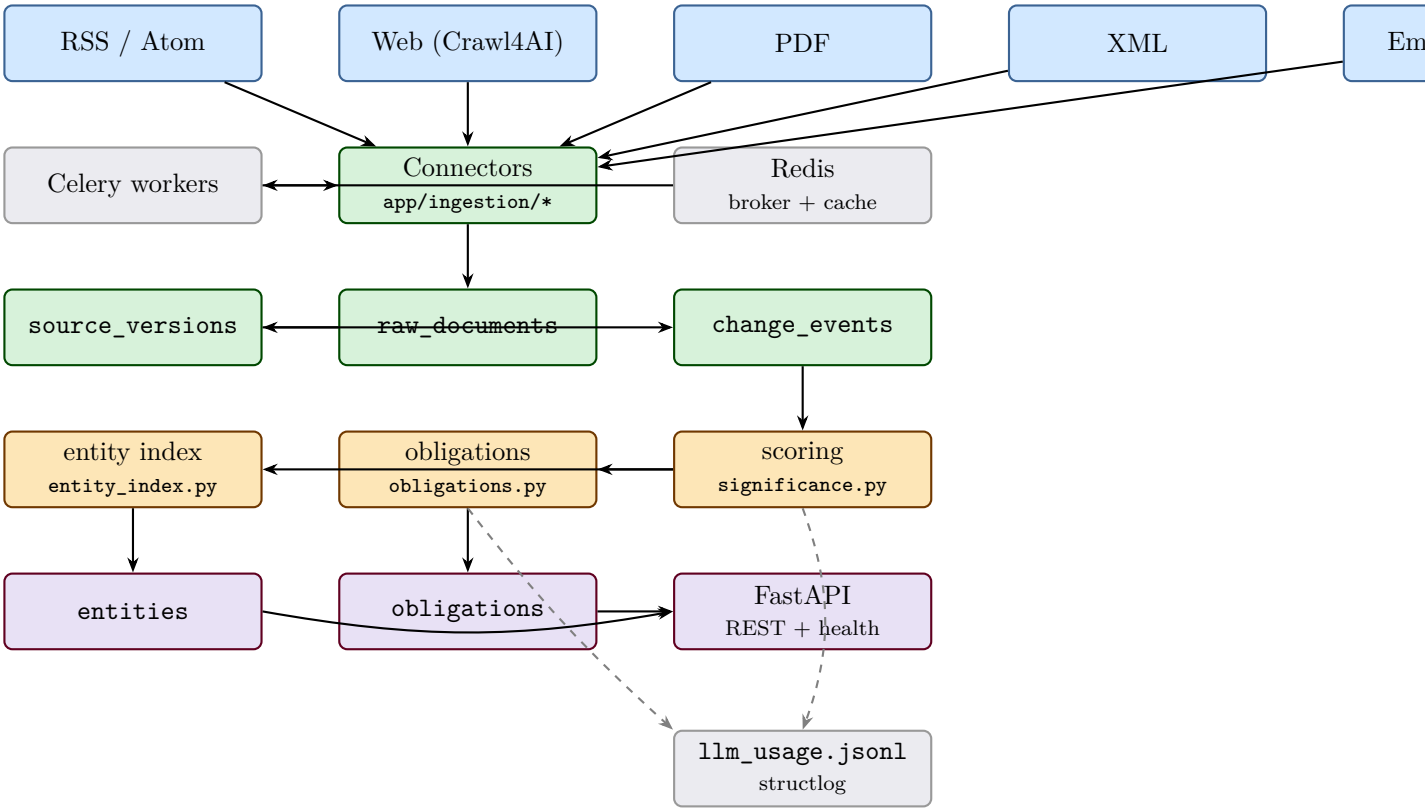
Regulatory Watch replaces that with an always-on pipeline that answers three questions for every piece of regulatory content:

1. **Has anything changed since the last check?** — deterministic, content-hash based.
2. **Does the change actually matter?** — LLM-scored on a 0–1 significance scale with a change-type taxonomy (*typo / minor wording / clarification / substantive / critical*).
3. **What concrete obligations does it impose?** — structured rows of *(actor, action, condition, deadline, penalty)* extracted from the text and persisted to the DB.

Downstream applications (alerts, dashboards, personalised digests, integrations with risk-management systems) read from those same structured tables — they never have to parse HTML themselves.

2 High-level architecture

2.1 Component map



2.2 Three layers, one pipeline

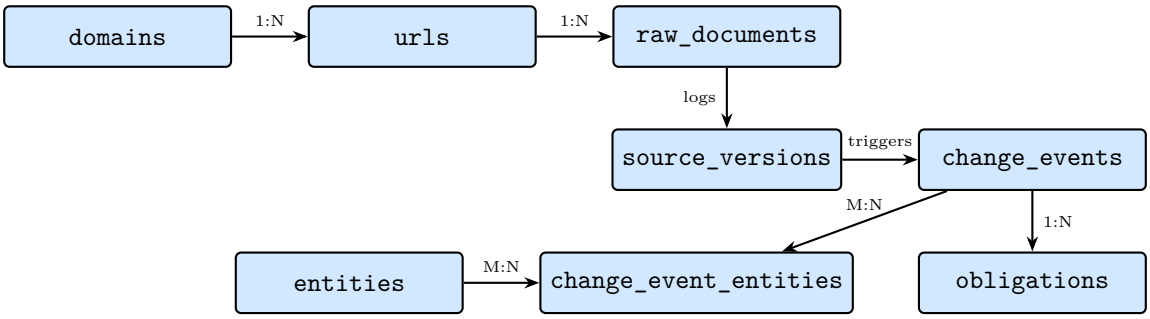
Layer	Responsibility
L1 Ingestion	Poll / crawl each source on a schedule. Normalise text, de-duplicate, persist `RawDocument`. One connector per channel. Deterministic, no LLM.
L2 Versioning & change detection	For every unchanged-URL re-fetch, compute SHA-256 on the normalised text. If it differs from the previous `SourceVersion`, persist the new version and emit a `ChangeEvent`. Still deterministic, no LLM.
L3 Significance & enrichment	Ask an LLM the compliance-officer question: *does this matter?* Record score (0–1), change-type, topic, affected entities, short summary.
L4 Obligations & entity index	For high-significance events (*score* ≥ 0.6), extract structured obligations (actor, action, deadline, penalty, type). Sync entity index across events.

Each layer is designed so the one below it can fail without blocking the one above: a stale LLM doesn't stop ingestion, a broken connector doesn't stop scoring for other sources.

3 Data model

Table	Purpose
`domains`	Seed registry: one row per regulator, holds rate-limit & schedule policy.
`urls`	Discovered URLs under a domain with priority, relevance and back-off state.
`fetch_runs`	One row per scheduled crawl (success / fail / counts).
`fetch_attempts`	One row per URL within a run (HTTP status, content hash, timings).
`raw_documents`	Latest content per source URL (web / pdf / rss / xml / email).
`source_versions`	Append-only version log keyed by `(source_url, content_hash)`.
`change_events`	One row per detected change; carries LLM score, topic, summary.
`entities`	Canonical entity registry (agencies, regulations, codes...).
`change_event_entities`	Many-to-many join of events ↔ entities.
`obligations`	Structured *(actor, action, deadline, penalty, type)* rows.

3.1 Relationships (simplified)



4 The pipeline in detail

4.1 Stage 1 — Ingestion (L1)

Five connectors, one per channel, all under `app/ingestion/`:

Connector	How it works
`WebConnector`	Crawl4AI (headless browser w/ JS) + BFS; fallback to httpx + BeautifulSoup. Anti-bot / blocker detection in 5 languages.
`PDFConnector`	`pdfplumber`; captures per-page offsets for citation.
`XMLConnector`	`lxml`; streaming for USLM/bulkdata-style XML.
`RSSConnector`	`feedparser`; one item → one document.
`EmailConnector`	IMAP; reads regulatory mailing lists (bulletins, press-releases).

Content extraction (`web_extractor.py`) strips boilerplate, navigation, cookie banners, and produces a stable text shape for downstream hashing.

4.2 Stage 2 — Versioning & change detection (L2)

Every fetch is normalised then SHA-256 hashed. The loop is:

1. Look up the latest `SourceVersion` for this URL.
2. If the hash is identical ⇒ bump `last_seen_at`, no event.
3. If the hash differs ⇒ insert new `SourceVersion`, emit a `ChangeEvent` with diff stats (added chars, removed chars, diff kind, unified diff preview), enqueue scoring.

All three paths run under a single Postgres transaction per document so the store is always consistent.

4.3 Stage 3 — Significance scoring (L3)

`app/services/significance.py` posts the diff + content to OpenAI in JSON mode and expects back:

```
{
  "score": 0.75,
  "change_type": "substantive",
  "topic": "financial_services",
  "affected_entities": ["Financial Conduct Authority", "PSR"],
  "deadline_changes": ["2026-06-30"],
  "summary": "FCA raises minimum capital thresholds..."
}
```

Change-type taxonomy (scale banded to the 0–1 score):

Band	Type	Meaning
0.00–0.19	`typo_or_cosmetic`	Whitespace, broken links, punctuation
0.20–0.39	`minor_wording`	Rephrasing with no legal effect
0.40–0.59	`clarification`	Explains existing rule more clearly
0.60–0.79	`substantive`	Rule, threshold or scope altered
0.80–1.00	`critical`	New deadline / penalty / obligation

Topic taxonomy: `customs_trade`, `financial_services`, `data_privacy`, `environmental`, `healthcare_pharma` `sanctions_export_control`, `labor_employment`, `tax_accounting`, `consumer_protection`, `corporate_govern` `other`.

Everything is fail-soft: on LLM error, the event gets `llm_error` set, a char-count heuristic can still drive alerts, and the circuit breaker (see below) pauses further calls.

4.4 Stage 4 — Obligations & entity index (L4)

For events with *score* ≥ 0.6, `obligations.py` asks the LLM to extract atomic compliance facts. Output is validated against a Pydantic schema and persisted one row per obligation:

Column	Example	Notes
`actor`	"firms offering BNPL agreements"	
`action`	"Extend promotional period. . . "	
`condition`	"if customer affected by Covid-19"	
`deadline_text`	"by 30 June 2026"	free-te
`deadline_date`	`2026-06-30`	parsed
`penalty`	"up to £1m per breach"	
`obligation_type`	`reporting` / `prohibition` / `threshold` / `disclosure` / `registration` / `penalty` / `other`	CHEC

`entity_index.py` normalises each entity mentioned by the LLM into a canonical form ("FCA" ≡ "Financial Conduct Authority") so cross-event queries (*"show all high-score changes touching the FCA in the last 30 days"*) are one SQL away.

5 Resilience & observability

Concern	Mechanism
Failing LLM	Per-task retry budget + Redis-backed circuit breaker (10 errors / 60 s → 5-min cooldown).
Overloading OpenAI	Celery rate limits: scoring 60/min, obligations 30/min.
Prompt regressions	Every call logs `request_hash`, `latency_ms` and raw-response on parse failure.
Cost drift	`llm_usage.jsonl` ledger with per-call USD cost; CLI report (`scripts/llm_cost_report.py`).
DB schema drift	Alembic is the sole schema writer; `create_all()` gated behind a dev flag.
Connection pool exhaustion	Explicit SQLAlchemy pool sizing; health checks.
Production secrets	Env-vars from vault; `.env` is dev-only (see `docs/secrets.md`).
Log aggregation	`structlog` JSON across API, Celery and services — parseable by Loki, Datadog, CloudWatch.
Anti-bot pages	Multilingual detector + Redis per-domain block counters.

6 API & operational surface

6.1 REST API (FastAPI, `/docs`)

Method	Path	Purpose
GET	`/health`	Liveness probe
GET	`/health/db`	DB + Redis health
POST	`/domains`	Register a new regulator source
GET	`/domains`	List registered domains
GET	`/domains/{id}`	Get one domain + policy
PATCH	`/domains/{id}`	Update rate-limit / status
DELETE	`/domains/{id}`	Remove a domain

6.2 Celery tasks (auto-scheduled by Celery-beat)

Task	Trigger	Period
`heartbeat`	Every 60 s	liveness
`rss_ingest_task`	Per registered feed	30 min
`web_crawl_task`	Per registered seed	6 h
`pdf_ingest_task`	Ad-hoc / discovered	on demand
`xml_ingest_task`	Per bulkdata source	24 h
`email_ingest_task`	IMAP mailbox	15 min
`score_change_event`	Auto-enqueued on diff	–
`extract_obligations_task`	After scoring (≥ 0.6)	–

6.3 Operator CLIs

Script	Shows
`scripts/show_changes.py`	Recent change events with score, type, topic, URL.
`scripts/show_entities.py`	Top entities by mention count (filter by type / score / window).
`scripts/show_obligations.py`	Structured obligations with deadlines & actors.
`scripts/score_backlog.py`	Re-score unscored / errored events (`-retry-errors`).
`scripts/llm_cost_report.py`	Token + USD report from the ledger.
`scripts/domain_report.sql`	Per-domain roll-up of fetches, changes, obligations.

7 Tech stack

Layer	Tool	Why
Web API	FastAPI	Pydantic-native, async, OpenAPI out of the box.
Async workers	Celery + Redis	Battle-tested, per-task rate limits, good ecosystem.
Scheduler	Celery-beat	Co-located with workers, no extra process.
ORM	SQLModel (SQLAlchemy + Pydantic)	Same model for DB and API.
DB	PostgreSQL 16	JSONB for entities, CHECK constraints on obligations.
Schema migrations	Alembic	Sole schema writer (no `create_all`).
Web crawling	Crawl4AI (Playwright)	Renders JS, markdown output.
HTML fallback	httpx + BeautifulSoup	Lightweight, JS-free pages.
PDF	pdfplumber	Per-page offsets (citations).
XML / feeds	lxml + feedparser	Stream-friendly.
LLM	OpenAI `gpt-4o-mini`	JSON mode, cheap, deterministic.
Logging	structlog	JSON for aggregators.
Tests	pytest + freezegun + fakeredis	93 unit tests, ∼0.65 s.
Deploy	Docker Compose	`api`, `worker`, `flower`, `db`, `redis`, `kafka`, `zookeeper`.

8 Economics

Marginal cost per event (measured from a live run):

Event type	Tokens	USD
Low-significance (scored, no obligations)	∼2,100	$0.000346
High-significance (scored + obligations)	∼4,200	$0.000725
1,000 events / month		$0.72
10,000 events / month		$7.25
100,000 events / month		$72.50

The LLM is a rounding error compared to crawler and database infra costs even at moderate scale. The 0.6 score-gate cuts the obligations spend by ∼60% and hard token caps (`LLM_MAX_*` in config) prevent a single runaway document from blowing the budget.

9 Current status

- **M1 — single-source ingestion:** complete.
- **M2 — multi-source ingestion (5 connectors):** complete.
- **M3 — versioning + change detection + LLM scoring:** complete.
- **M4 — obligations + entity index (LLM-only):** complete.

- **Professionalism pass:** complete (12/12 items, 93 unit tests passing).
- **Token & cost observability:** complete (ledger + CLI report + per-call USD).

Polish items queued (not blocking pilot): CI (GitHub Actions) running the test suite on every push; Prometheus metrics + `/metrics` endpoint; end-to-end integration test covering crawl → score → obligations in one run; prompt refinement so past-meeting dates aren't misread as compliance deadlines.

Source: `regulation-prj-v1` — see `docs/product_overview.tex` to regenerate this document.